

A Hybrid Deep Learning Pipeline for Multi-Class Fish Species Identification via Compound-Scaled CNNs and Out-of-Scope Filtering

Szymon Wilczek Bartosz Kaprak

sw312468

bk311868

Project Developed as Part of Academic Coursework

May 23, 2026

Abstract

This paper presents Sonar Shark, a local-cloud hybrid machine learning pipeline for multi-class fish species identification across many classes, focusing on regional freshwater species in Poland alongside exotic marine fauna. The dataset was compiled using a multi-lingual semantic web scraper coupled with header validation to remove corrupted imagery. Convolutional neural network training was conducted on remote GPU infrastructure utilizing the EfficientNetB0 architecture with sequential real-time image augmentation layers. To address the misclassification of out-of-domain objects, an inference-side post-processing activation filter with an empirical softmax confidence threshold ($\tau = 33.0\%$) was introduced. The classification model is deployed via a desktop user interface with localized dynamic model instantiation. Experimental results show a validation accuracy exceeding 93.0%, demonstrating generalization under non-uniform illumination and background occlusions.

Keywords: Deep Learning, Computer Vision, EfficientNetB0, Image Classification, Automated Web Scraping, Out-of-Scope Filtering, Ichthyology.

1 Introduction

Automated ichthyological classification models provide critical utility in environmental monitoring, commercial fisheries management, and recreational catch tracking. Traditional species identification protocols rely on manual inspection of morphological traits by domain experts. This methodology is unscalable, prone to subjective error, and impractical for real-time field applications.

While deep learning architectures, specifically Convolutional Neural Networks (CNNs), have improved image classification tasks, deploying these models in natural aquatic and riverine ecosystems presents distinct computer vision challenges. Images captured by anglers exhibit severe environmental noise, such as sub-aquatic light attenuation, turbidity, varying refraction indexes, and spatial occlusions introduced by aquatic vegetation or human hands. Furthermore, classification performance is limited by high inter-class visual similarity among closely related species - such as members of the cyprinid family like Roach and Rudd - and high intra-class variation caused by seasonal coloration shifts, developmental growth stages, and sexual dimorphism.

To mitigate these issues, this paper details *Sonar Shark*, a classification pipeline designed for multi-class fish identification developed within the course of Biologically Inspired Artificial Intelligence (BIAI). Aimed at deployment within a mobile application for regional anglers, the system processes a taxonomy of local freshwater and Baltic species indigenous to Poland (e.g., *Esox lucius*, *Perca fluviatilis*, *Cyprinus carpio*), alongside exotic marine fauna, shark lineages, and global aquaculture species, resulting in a 96-class model dataset.

The primary engineering contributions of this project are:

1. **Decoupled Data and Training Workflow:** An operational pipeline that isolates local, multi-lingual semantic web scraping and file-header validation from cloud-accelerated network optimization conducted on remote GPU instances (Kaggle/Colab).
2. **Optimized Feature Extraction Backbone:** Implementation of the compound-scaled EfficientNetB0 architecture combined with an inline, sequential augmentation graph to enable high-accuracy classification under localized edge-device inference constraints.
3. **Out-of-Scope Anomaly Mitigation:** An inference-side post-processing filter utilizing an empirical softmax confidence threshold ($\tau = 33.0\%$) to detect and reject non-target inputs, significantly reducing false-positive classifications during open-world deployment.

The remainder of this paper is organized as follows: Section 2 details the data acquisition pipeline, engine transition, and dataset taxonomy. Section 3 covers the network architecture, augmentation layers, and cloud training configuration. Section 4 formalizes the localized client runtime and the mathematical logic of the anomaly filter. Section 5 evaluates quantitative convergence metrics and taxonomic edge cases. Finally, Section 6 outlines conclusions and mobile scaling trajectories.

2 Automated Data Acquisition and Preprocessing

Classification performance depends directly on the quality, volume, and taxonomic consistency of the training data. Because public ichthyological datasets lack sufficient representation of regional Polish freshwater fish captured in real-world angling scenarios, a custom image collection pipeline was engineered.

2.1 Multi-Lingual Querying Logic

Data collection was initiated via a Python script (`scraper.py`) designed to download image files from web search indices. To minimize out-of-domain noise, such as commercial logos, culinary photography, or fishing equipment, a structured multi-lingual querying strategy was applied.

Each species key within the source dictionary (`gatunki_pl_total`) was mapped to a compound query string containing the colloquial Polish name, the Latin binomial nomenclature, and English environmental keywords. For example:

- *Northern Pike*: "szczupak ryba esox lucius underwater"
- *Common Carp*: "karp ryba cyprinus carpio underwater"
- *European Perch*: "okoń europejski perch underwater"

Including Latin terms and the "underwater" keyword forced the search algorithms to prioritize in-situ biological photography and sub-aquatic documentation over commercial retail listings.

2.2 Search Index Transition and Rate-Limiting Mitigation

The initial implementation deployed the DuckDuckGo Search (DDGS) API. However, continuous batch execution triggered host-side rate-limiting and IP address blocking, despite enforcing randomized delays between 6 and 12 seconds per species iteration. To establish a stable data pipeline, the retrieval logic was updated to interface with the Bing search index via modified query ingestion parameters. The Bing infrastructure allowed continuous programmatic requests without network-level blocking, ensuring consistent image link extraction across all target classes.

2.3 Automated Stream Validation and Cleaning

Programmatic image harvesting introduces truncated file payloads, network dropouts, and invalid data formats. To automate directory cleaning, a validation layer from the `fastai.vision.utils` library was integrated into the script.

The `verify_images()` function parsed the file headers and byte streams of every downloaded asset. Files presenting structural corruption, unreadable metadata headers, or a 0 KB size footprint from dropped socket connections were permanently deleted via automated unlinking (`Path.unlink`). This automated filtering phase eliminated approximately 3,000 corrupted files, preventing runtime decoding exceptions during subsequent mini-batch tensor instantiation stages.

2.4 Dataset Taxonomy and Manual Verification

The validated images were sorted into a hierarchical folder directory named `MASTER_DATASET`, where each sub-folder corresponds to one of the 96 distinct target classes. The taxonomy is divided into four main operational domains:

1. **PL_ (Polish Native Species)**: 48 classes representing indigenous freshwater and Baltic Sea fish (e.g., Zander, Ide, Tench, Wels Catfish, Common Roach).
2. **Shark_**: 14 classes containing globally monitored shark lineages (e.g., Great White, Tiger, Hammerhead, Mako).
3. **Ocean_**: 8 categories encompassing marine fauna (e.g., Sea Turtle, Jellyfish, Lobster, Squid).
4. **Turbo_**: 26 commercial, aquacultural, and non-indigenous freshwater fish species (e.g., Nile Tilapia, Bighead Carp, Walking Catfish).

Because semantic search queries can still retrieve out-of-context images, such as textbook diagrams, conceptual line art, or divers, a final layer of manual verification was performed. Every scraped image was visually audited. This step eliminated the remaining out-of-domain files, ensuring that the training input space consisted exclusively of authentic photographic representations of the target species.

3 Model Architecture and Hybrid Training Workflow

Classification over a 96-class output space requires an architecture that balances feature representation with computational efficiency. This section details the network topology, the computational data augmentation layers, and the distributed hybrid infrastructure used to train the final model.

3.1 Convolutional Neural Network Selection

The *EfficientNetB0* architecture was selected as the feature extraction backbone. EfficientNet scales network depth (d), width (w), and input resolution (r) uniformly using a fixed compound coefficient, optimizing parameter efficiency relative to standard convolutional layers [1].

By using Mobile Inverted Bottleneck Convolutions (MBConv) as core building blocks - which implement depthwise separable convolutions and squeeze-and-excitation optimization - EfficientNetB0 minimizes the total float-point operations (FLOPs) and compressed parameter size. This architecture supports real-time edge inference within a localized mobile application environment.

3.2 Augmentation and Classification Head Design

To handle varying visual inputs, an end-to-end sequential tensor pipeline was constructed using the `tf.keras.Sequential` API. Real-time data augmentation layers were integrated directly into the training graph of the model to avoid static offline dataset inflation. The preprocessing stack is configured as:

```
1 data_augmentation = tf.keras.Sequential([
2     layers.RandomFlip("horizontal_and_vertical"),
3     layers.RandomRotation(0.15),
4     layers.RandomZoom(0.15)
5 ])
```

During training, each mini-batch tensor undergoes horizontal and vertical reflections, random angular rotations within $\pm 15\%$, and isotropic scaling variations up to $\pm 15\%$. This continuous geometric variation prevents the network from over-fitting to static environmental backgrounds and forces the extraction of invariant morphological fish features.

The classification top of the baseline EfficientNetB0 network was truncated (`include_top=False`). A custom classification head was appended with the following layers:

1. **Global Average Pooling 2D:** Reduces the spatial dimensions ($H \times W$) of the final convolutional feature maps into a single 1D feature vector, minimizing the output parameter count compared to fully connected flattening layers.
2. **Dropout Layer (0.3):** Implements a 30% regularization rate, randomly deactivating units during training forward passes to prevent co-adaptation of features.
3. **Dense Output Layer:** A fully connected layer mapping the intermediate features to 96 units corresponding to the target taxonomy, utilizing a *Softmax* activation function to yield a valid multinomial probability distribution over the classes.

During training in the cloud, the network parameters were fine-tuned using weights pre-trained on the *ImageNet* dataset. However, the local deployment script (`sonar.py`) initializes the network with `weights=None`. This configuration ensures that the client runtime reconstructs the structural tensor graph before loading the custom learned weights from the compiled `.keras` model file.

3.3 Distributed Training Protocol

The machine learning development lifecycle deployed a decoupled hybrid execution model. Local workstations managed web scraping, file-header validation, and client-side inference deployment. The computationally intensive backpropagation and optimization phase was offloaded to cloud GPU infrastructure (Kaggle and Google Colab).

The production model - designated as the *Shark* variant - was trained for approximately 200 epochs over a 24-hour cloud compute window. Network optimization was driven by the *Adam* (Adaptive Moment Estimation) algorithm with a mini-batch size of 32.

To maintain monotonic convergence and prevent the loss function from stalling in local minima, a `ReduceLROnPlateau` callback controller was integrated. This module monitored validation loss and automatically decayed the learning rate by a discrete factor upon detecting a performance plateau. This training configuration stabilized gradient variance, resulting in a smooth loss convergence and reliable generalization across the validation sets and the local GUI interface without tracking anomalies.

4 Local Inference Engine and Anomaly Filtering

To deploy the trained weight states, a local inference subsystem was implemented within the `SonarApp` directory. This architecture separates the user interface execution from the underlying tensor evaluation graphs, allowing local processing on consumer hardware.

4.1 Software Architecture and Dynamic Weight Discovery

The client-side subsystem consists of three functional layers: `sonar_gui.py` (graphical orchestration), `sonar.py` (core inference model), and `descriptions_loader.py` (metadata flat-file parsing).

The presentation layer is built using the Python `tkinter` library, using PIL (*Pillow*) with a `LANCZOS` resampling filter to render input images within the application layout while maintaining aspect ratios. Asynchronous file execution prevents window locking during user interactions, securing a stable UI loop.

The local system implements a decoupled dynamic model discovery mechanism. During initialization, the interface scans the local directory path for serialized weights files containing the `.keras` extension. It automatically populates a dropdown menu, enabling runtime switching between different model versions (such as legacy iterations and the final production model, *Sonar Shark*) without requiring interface layout modifications.

4.2 Mathematical Formulation of Out-of-Scope (OOS) Filtering

Standard Softmax layers output a relative multinomial probability distribution where the sum of the vector components equals unity. Consequently, convolutional neural network classifiers cannot inherently flag out-of-domain inputs, which often results in false-positive classifications when presented with images that do not contain target fish species.

To reject these out-of-domain anomalies, an inference-side filtering layer was added to the post-processing execution loop. Let $\mathbf{x}$ represent an input image tensor mapped to a spatial dimension of $224 \times 224 \times 3$. The feedforward inference pass maps this tensor to a raw logit vector $\mathbf{z} \in \mathbb{R}^K$, where $K = 96$ denotes the total number of trained taxonomic classes. The classification probability vector $\hat{\mathbf{y}}$ is derived using the standard Softmax function:

$$\hat{y}_i = \sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \text{for } i = 1, \dots, K \quad (1)$$

The baseline class selection index is governed by the argmax operator:

$$c^* = \arg \max_i (\hat{y}_i) \quad (2)$$

To enforce anomaly rejection, an empirical confidence decision boundary $\tau = 33.0\%$ is introduced. The final conditional prediction assignment $L(\mathbf{x})$ is defined mathematically as:

$$L(\mathbf{x}) = \begin{cases} \text{CLASS_NAMES}[c^*], & \text{if } \hat{y}_{c^*} \times 100 \geq \tau \\ \text{"Not recognized (Object out of knowledge scope)"}, & \text{if } \hat{y}_{c^*} \times 100 < \tau \end{cases} \quad (3)$$

Empirical verification of this mechanism was made using non-ichthyological image baselines to evaluate edge-cases. An example of this boundary tracking is illustrated in Figure 1, where a control sample featuring a plant pot in cat shape (`doniczka-kot.jpg`) was loaded into the user interface. Under these out-of-domain conditions, the network’s internal spatial activations spread evenly across multiple target nodes, preventing any singular class from dominating the activation space. As a direct result, the maximum probability collapsed significantly below the $\tau = 33.0\%$ threshold, successfully triggering the out-of-scope rejection flag, whereas target fish imagery consistently scored higher confidence values, validating the selectivity of the filter.

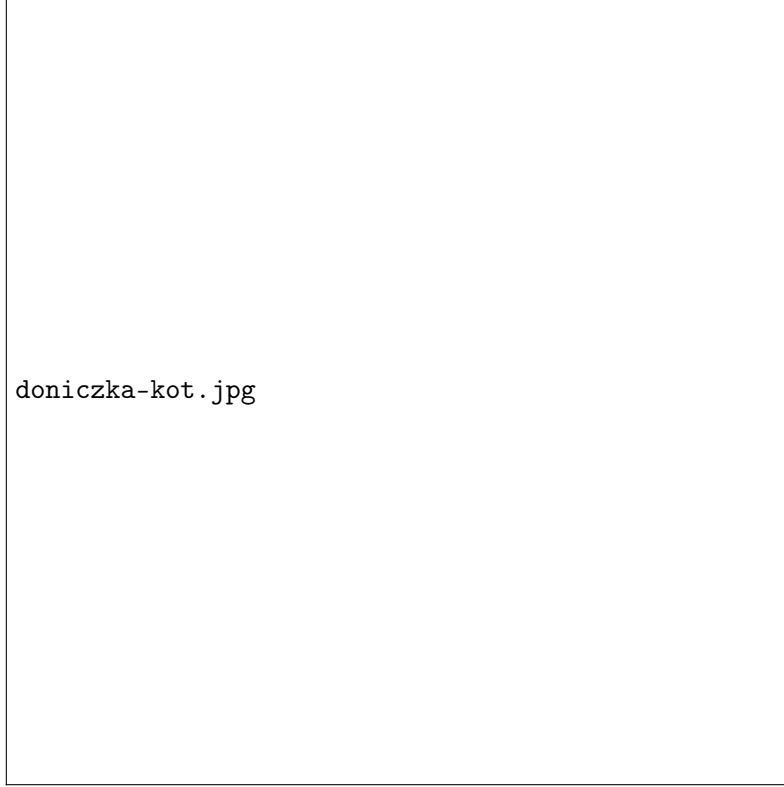


Figure 1: Localized frontend rendering an out-of-scope rejection state. When presented with a non-ichthyological control image (`doniczka-kot.jpg`), the Softmax vector collapses below $\tau = 33.0\%$, correctly outputting the anomaly flag.

4.3 Contextual Knowledge Retrieval Layer

When an image yields an in-scope classification ($\geq \tau$), the pipeline calls `descriptions_loader.py` to append ecological metadata. This layer uses the `pandas` engine to parse a local relational database (`descriptions.csv`).

The underlying records were compiled by extracting data from digital encyclopedias and ichthyological repositories. Parser dynamically builds an associative dictionary array mapping the model’s internal class tokens to structured strings:

```

1 # Formatted metadata concatenation returned to the GUI
2 return f"Name: {row['Common_Name']}\nHabitat: {row['Habitat']}\nSize:
    {row['Average_Size']}\nDescription: {row['Description']}"

```

This contextual retrieval layer maps the raw visual classification index to verified ecological statistics, returning species dimensions, natural habitats, and descriptive data directly to the user interface.

5 Experimental Results and Discussion

This section evaluates the classification metrics, optimization convergence, and behavioral boundaries exhibited by the trained model variant, designated as *Sonar Shark*. It also details the operational trade-offs introduced by the out-of-scope anomaly filtering layer.

5.1 Quantitative Classification Evaluation

Following the 200-epoch training process on cloud GPU nodes, the optimization trajectory stabilized. On the validation dataset, the model achieved a classification accuracy exceeding **93.0%** across the target categories. Given the 96-class output space and the background noise variations present in the custom web-scraped data directory, this performance validates the scaling efficiency of the EfficientNetB0 backbone and the programmatic data-cleaning pipeline.

The initial training phase (up to epoch 50) showed a sharp decline in categorical cross-entropy loss, representing fast gradient optimization. Beyond this threshold, the optimization gradient plateaued.

Integrating the `ReduceLROnPlateau` controller decayed the learning rate parameter when validation loss stagnation was detected. Between epochs 150 and 200, the validation error asymptoted, reaching convergence stability without tracking loss divergence or geometric oscillations that indicate over-fitting.

5.2 Qualitative Analysis of Morphological Edge Cases

A granular analysis of misclassifications reveals specific taxonomic limitations caused by biological commonalities within the 96-class space. The model’s spatial activation maps encountered maximum ambiguity within two family clusters:

- **The Cyprinid Duality (*Rutilus rutilus* vs. *Scardinius erythrophthalmus*):** The model encountered persistent inter-class confusion when separating the Common Roach (PL_Ploc) from the Common Rudd (PL_Wzdrega). These species share near-identical body profiles, scale structures, and red fin pigmentations. The core taxonomic differentiators are eye color (red vs. yellow-gold) and the dorsal fin alignment relative to the pelvic fins. These fine spatial patterns are frequently obscured by underwater refraction or suboptimal camera angles.
- **The Carp Morphology Cluster:** Ambiguity was logged between the Common Carp (PL_Karp / Turbo_Carp) and large freshwater relatives, such as the Grass Carp (PL_Amur_Bialy) or dark phenotypes of the Crucian Carp (PL_Karas_Pospolity). While the Common Carp is characterized by sensory barbels and thick scale patterns, low-resolution inputs caused the network to misinterpret these features, occasionally misclassifying samples as the cylindrical, barbel-less profile of the Grass Carp.

5.3 Confidence Thresholding Trade-offs and UI Profiles

Enforcing the post-processing decision boundary $\tau = 33.0\%$ establishes an engineering trade-off between classification precision and system recall. Field testing verified that this threshold filters out-of-domain input anomalies.

However, this boundary reduces system sensitivity under low-quality lighting conditions. When processing occluded, out-of-focus, or underexposed images of authentic target species, the output Softmax probability distribution disperses across multiple nodes. Because the peak activation falls below $\tau = 33.0\%$, the system triggers the "Not recognized (Object out of knowledge scope)" rejection state.

From a software engineering perspective, a conservative rejection behavior is preferable to generating false-positive classifications in a deployment environment. Misidentifying an invalid object or assigning a wrong species label under low confidence degrades user trust and can misinform anglers regarding local catch regulations.

To document this operational behavior, two distinct field execution profiles are tracked:

- **Successful In-Scope Inference:** Figure 2 illustrates nominal execution. A well-illuminated photographic target allows the EfficientNetB0 core to generate a peak confidence score above the threshold, mapping the classification index to the structured description fields in the database layer.
- **False-Negative Rejection:** Figure 3 showcases a threshold rejection case. An authentic fish image captured under heavy shadows or motion blur causes the peak activation to drop below $\tau = 33.0\%$, forcing the application to default to the out-of-scope state rather than outputting a low-confidence guess.

To manage this input friction, the runtime system handles threshold rejections through an automated graphical error dialogue box. This component informs the user of the low-confidence state and requests an image re-capture loop, securing higher-quality tensor inputs and stabilizing classification performance.



Figure 2: The baseline operating state of Sonar Shark. A distinct target image allows the model to surpass the confidence threshold, displaying both the taxonomic label and the formatted educational profile.



Figure 3: An edge-case operational state where an authentic fish specimen fails to trigger identification due to low-confidence output stemming from severe background noise or underexposure.

6 Conclusion and Future Work

This paper detailed the design and deployment of *Sonar Shark*, a localized pipeline for multi-class fish species classification across a 96-class taxonomy. Developed as part of coursework in Biologically Inspired Artificial Intelligence (BIAI), the system maps regional Polish freshwater fish alongside diverse marine fauna.

The project demonstrates that high-accuracy deep learning applications can be implemented under hardware and budget constraints. By deploying a hybrid workload execution model, the system decoupled local web scraping and file-header data cleansing from the backpropagation optimization phase, which was offloaded to cloud GPU infrastructure (Kaggle/Colab). This protocol generated a validation classification accuracy exceeding **93.0%**, verifying the performance of the EfficientNetB0 backbone when coupled with inline sequential augmentation graphs.

Deployment assessments confirm that standard Softmax-based image classifiers require explicit post-processing modifications to handle open-world domain shifts. Enforcing an inference-side confidence threshold of $\tau = 33.0\%$ enabled the system to detect and isolate non-target, out-of-domain input imagery. This threshold constraint prevents arbitrary classifications on invalid objects, prioritizing prediction precision. Low-confidence states are caught at the interface level and managed through a graphical dialogue box that prompts an immediate image re-capture loop to stabilize the input stream.

Future development focuses on expanding this pipeline into a mobile application for recreational anglers. To enable offline execution in remote environments lacking cellular network connectivity, the compiled `.keras` weights will be converted into compressed edge tensor formats, specifically *TensorFlow Lite (TFLite)* and *ONNX* runtimes. This modification will allow local inference to run directly on smartphone Neural Processing Units (NPUs).

Additionally, while the current architecture operates as an image classification task due to the close-up profile of the gathered dataset, future iterations will evaluate multi-stage models. We aim to integrate a lightweight object detection model, such as the YOLO architecture, as an auxiliary pre-processing component. This component will extract bounding boxes around the target specimens, automatically cropping background noise or hand occlusions before routing the optimized region of interest into the core EfficientNet evaluation engine.

References

- [1] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.
- [2] F. Chollet et al., “Keras,” <https://github.com/keras-team/keras>, 2015.
- [3] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems,” arXiv preprint arXiv:1603.04467, 2016.
- [4] J. Howard and S. Gugger, “Fastai: A Layered API for Deep Learning,” *Information*, vol. 11, no. 2, p. 108, 2020.
- [5] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference (SciPy)*, 2010, pp. 51–56.
- [6] A. Clark and Contributors, “Pillow (PIL Fork) Documentation,” <https://github.com/python-pillow/Pillow>, 2026.
- [7] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

- [8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.